

torch.frombuffer#

<https://pytorch.org/docs/stable/generated/torch.frombuffer.html>

Description:

Creates a 1-dimensional Tensor from an object that implements the Python buffer protocol. Skips the first offset bytes in the buffer, and interprets the rest of the raw bytes as a 1-dimensional tensor of type dtype with count elements. Note that either of the following must be true:

1. count is a positive non-zero number, and the total number of bytes in the buffer is more than offset plus count times the size (in bytes) of dtype.
2. count is negative, and the length (number of bytes) of the buffer subtracted by the offset is a multiple of the size (in bytes) of dtype.

Function Signature

```
torch.frombuffer(buffer, *, dtype, count=-1, offset=0,  
requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (`torch.dtype`) – the desired data type of returned tensor. `count` (int, optional) – the number of desired elements to be read. If negative, all the elements (until the end of the buffer) will be read. Default: -1. `offset` (int, optional) – the number of bytes to skip at the start of the buffer. Default: 0. `requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Code Examples

```
>>> import array
>>> a = array.array('i', [1, 2, 3])
>>> t = torch.frombuffer(a, dtype=torch.int32)
>>> t
tensor([ 1,  2,  3])
>>> t[0] = -1
>>> a
array([-1,  2,  3])

>>> # Interprets the signed char bytes as 32-bit integers.
>>> # Each 4 signed char elements will be interpreted as
>>> # 1 signed 32-bit integer.
>>> import array
>>> a = array.array('b', [-1, 0, 0, 0])
>>> torch.frombuffer(a, dtype=torch.int32)
tensor([255], dtype=torch.int32)
```

Notes & Warnings

NOTE

Note This function increments the reference count for the object that owns the shared memory. Therefore, such memory will not be deallocated before the returned tensor goes out of scope.

⚠ WARNING

Warning This function's behavior is undefined when passed an object implementing the buffer protocol whose data is not on the CPU. Doing so is likely to cause a segmentation fault.

⚠ WARNING

Warning This function does not try to infer the dtype (hence, it is not optional). Passing a different dtype than its source may result in unexpected behavior.

Detailed Documentation

Documentation

Creates a 1-dimensional Tensor from an object that implements the Python buffer protocol.

Skips the first offset bytes in the buffer, and interprets the rest of the raw bytes as a 1-dimensional tensor of type dtype with count elements.

Note that either of the following must be true:

1. `count` is a positive non-zero number, and the total number of bytes in the buffer is more than `offset` plus `count` times the size (in bytes) of `dtype`.
2. `count` is negative, and the length (number of bytes) of the buffer subtracted by the `offset` is a multiple of the size (in bytes) of `dtype`.

The returned tensor and buffer share the same memory. Modifications to the tensor will be reflected in the buffer and vice versa. The returned tensor is not resizable.

This function increments the reference count for the object that owns the shared memory. Therefore, such memory will not be deallocated before the returned tensor goes out of scope.

This function's behavior is undefined when passed an object implementing the buffer protocol whose data is not on the CPU. Doing so is likely to cause a segmentation fault.

This function does not try to infer the `dtype` (hence, it is not optional). Passing a different `dtype` than its source may result in unexpected behavior.

`buffer` (object) – a Python object that exposes the buffer interface.

`dtype` (`torch.dtype`) – the desired data type of returned tensor.

`count` (int, optional) – the number of desired elements to be read. If negative, all the elements (until the end of the buffer) will be read. Default: -1.

`offset` (int, optional) – the number of bytes to skip at the start of the buffer. Default: 0.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

